

Sequential Task:

Threads - Multithreaded Robot Assembly Line

Object-Oriented Programming and GUI

May 11, 2026

1. Declare a class **Robot**, which represents a robot with a unique identifier, name, current status, and creation time.
 - (a) in the fields section implement
 - Add a private static field **counter** of type **int**, which will store the number of created robots.
 - Initialize the field with the value 0.
 - Add a private immutable field **id** of type **int**, which will store the robot identifier.
 - Add a private immutable field **name** of type **String**, which will store the robot name.
 - Add a private field **currentStatus** of type **String** marked as **volatile**, which will store the current robot status.
 - Ensure visibility of status changes between threads.
 - Add a private immutable field **createdAt** of type **long**, which will store the moment the robot was created.
 - (b) Implement the no-argument constructor **Robot()**.
 - Increase the value of the static field **counter** by 1.
 - Assign the new counter value to the **id** field.
 - Create the robot name in the format **"Robot-XXX"**.
 - Use **String.format("%03d", id)** to generate a three-digit number with leading zeros.
 - Combine the prefix **"Robot-"** with the generated number.
 - Assign the created name to the **name** field.
 - Set the initial robot status to **"CREATED"**.
 - Retrieve the current system time using **System.currentTimeMillis()**.
 - Assign the obtained value to the **createdAt** field.
 - (c) Implement the **getId()** method.
 - i. Return the value of the **id** field.
 - (d) Implement the **getName()** method.

- i. Return the value of the `name` field.
 - (e) Implement the `getCurrentStatus()` method.
 - i. Return the value of the `currentStatus` field.
 - (f) Implement the `setCurrentStatus(String status)` method.
 - i. Accept the parameter `status`.
 - ii. Assign the passed value to the `currentStatus` field.
 - (g) Implement the `getCreatedAt()` method.
 - i. Return the value of the `createdAt` field.
 - (h) Override the `toString()` method.
 - i. Return the robot name stored in the `name` field.
2. Declare a class `Buffer`, which represents a synchronized FIFO buffer storing `Robot` objects with limited capacity.
- (a) Add a private immutable field `queue` of type `Queue<Robot>`, which stores robots waiting in the buffer.
 - (b) Add a private immutable field `capacity` of type `int`, which specifies the maximum number of elements in the buffer.
 - (c) Add a private immutable field `name` of type `String`, which stores the buffer name.
 - (d) Add a private field `closed` of type `boolean`, which specifies whether the buffer has been closed.
 - (e) Implement the constructor `Buffer(String name, int capacity)`.
 - i. Assign the parameter `name` to the `name` field.
 - ii. Assign the parameter `capacity` to the `capacity` field.
 - iii. Create a new `LinkedList<>` queue and assign it to the `queue` field.
 - iv. Set the `closed` field to `false`.
 - (f) Implement the synchronized method `put(Robot robot)`, which adds a robot to the buffer.
 - i. While the queue size is greater than or equal to `capacity` and the buffer is not closed, print a message about the full buffer and call `wait()`.
 - ii. After waiting finishes, check whether the buffer has been closed.
 - iii. If the buffer is closed, throw an `InterruptedException` with a message containing the buffer name.
 - iv. Add the `robot` object to the queue by calling `queue.add(robot)`.
 - v. Set the robot status to `"WAITING_IN_" + name` by calling `robot.setCurrentStatus(...)`.
 - vi. Print a message about adding the robot to the buffer together with the current queue size and capacity.
 - vii. Call `notifyAll()` to resume threads waiting for a change in the buffer state.

- (g) Implement the synchronized method `take()`, which retrieves a robot from the buffer.
 - i. While the queue is empty and the buffer is not closed, print a message about the empty buffer and call `wait()`.
 - ii. If the queue is empty and the buffer is closed, return `null`.
 - iii. Retrieve and remove the first robot from the queue by calling `queue.poll()`.
 - iv. Print a message about taking the robot from the buffer together with the current queue size and capacity.
 - v. Call `notifyAll()` to resume threads waiting for a change in the buffer state.
 - vi. Return the retrieved robot.
 - (h) Implement the synchronized method `close()`, which closes the buffer.
 - i. Set the `closed` field to `true`.
 - ii. Print a message informing that the buffer has been closed.
 - iii. Call `notifyAll()` to wake up all threads waiting on the buffer.
 - (i) Implement the synchronized method `size()`, which returns the current queue size.
 - i. Return the result of `queue.size()`.
 - (j) Implement the synchronized method `isClosed()`, which returns information about whether the buffer is closed.
 - i. Return the value of the `closed` field.
 - (k) Implement the `getName()` method, which returns the buffer name.
 - i. Return the value of the `name` field.
3. Declare a class `Stage`, which represents a production station processing objects of type `Robot`.
- (a) Add a private immutable field `name` of type `String`, which stores the station name.
 - (b) Add a private immutable field `processingTimeMin` of type `int`, which specifies the minimum robot processing time.
 - (c) Add a private immutable field `processingTimeMax` of type `int`, which specifies the maximum robot processing time.
 - (d) Add a private field `occupied` of type `boolean`, which specifies whether the station is currently occupied.
 - (e) Add a private field `currentRobot` of type `Robot`, which stores the robot currently being processed.
 - (f) Add a private field `processedCount` of type `int`, which stores the number of processed robots.
 - (g) Implement the constructor `Stage(String name, int processingTimeMin, int processingTimeMax)`.
 - i. Assign the parameter `name` to the `name` field.
 - ii. Assign the parameter `processingTimeMin` to the `processingTimeMin` field.

- iii. Assign the parameter `processingTimeMax` to the `processingTimeMax` field.
 - iv. Set the `occupied` field to `false`.
 - v. Set the `currentRobot` field to `null`.
 - vi. Set the `processedCount` field to 0.
- (h) Implement the synchronized method `acquire(Robot robot)`, whose task is to occupy the station by a robot.
- i. While the `occupied` field has the value `true`, put the current thread into the waiting state.
 - ii. Before entering the waiting state, print a message informing:
 - which station is occupied,
 - which robot currently occupies the station,
 - which robot is waiting to enter.
 - iii. Call the `wait()` method.
 - iv. After obtaining access, set the `occupied` field to `true`.
 - v. Assign the passed robot to the `currentRobot` field.
 - vi. Set the robot status to `"IN_STAGE_" + name`.
 - vii. Display a message informing that robot processing has started and that the station has been occupied.
- (i) Implement the `process(Robot robot)` method, whose task is to simulate robot processing.
- i. Calculate a random processing time.
 - Start with the value `processingTimeMin`.
 - Add a random value from the range from 0 to `processingTimeMax - processingTimeMin`.
 - Use `Math.random()`.
 - ii. Store the calculated time in the variable `time`.
 - iii. Display a message informing:
 - at which station processing is taking place,
 - which robot is being processed,
 - what the operation time is in milliseconds.
 - iv. Pause the execution of the current thread by calling `Thread.sleep(time)`.
 - v. Declare that the method may throw an `InterruptedException`.
- (j) Implement the synchronized method `release(Robot robot)`, whose task is to release the station after the work is completed.
- i. Set the `occupied` field to `false`.
 - ii. Set the `currentRobot` field to `null`.
 - iii. Increase the value of the `processedCount` field by 1.
 - iv. Set the robot status to `"DONE_" + name`.
 - v. Display a message informing:

- that the robot has finished processing,
 - that the station is free,
 - how many robots have been processed so far.
- vi. Call the `notify()` method to resume one of the waiting threads.
- (k) Implement the `getName()` method.
- i. Return the value of the `name` field.
- (l) Implement the synchronized method `getProcessedCount()`.
- i. Return the value of the `processedCount` field.
- (m) Implement the synchronized method `isOccupied()`.
- i. Return the value of the `occupied` field.
4. 🚩 Milestone 1: present the results of your work to the instructor
5. Declare a class `StageWorker`, which extends `Thread` and represents a worker handling one stage of the production line.
- (a) Add a private immutable field `stage` of type `Stage`, which stores the station handled by the worker.
 - (b) Add a private immutable field `inputBuffer` of type `Buffer`, which stores the input buffer with robots to be processed.
 - (c) Add a private immutable field `outputBuffer` of type `Buffer`, which stores the output buffer for processed robots.
 - (d) Add a private field `running` of type `boolean` marked as `volatile`, which controls the worker loop.
 - (e) Implement the constructor `StageWorker(String workerName, Stage stage, Buffer inputBuffer, Buffer outputBuffer)`.
 - i. Call the superclass constructor `Thread`, passing `workerName` as the thread name.
 - ii. Assign the passed values to the `stage`, `inputBuffer`, and `outputBuffer` fields.
 - iii. Set the `running` field to `true`.
 - (f) Override the `run()` method, which defines the full work of the thread.
 - i. Display a message informing that the worker has started and indicating the name of the handled stage.
 - ii. Place the main execution logic inside a `try` block.
 - iii. Start a `while` loop that runs as long as the `running` field has the value `true`.
 - iv. Retrieve a robot from the input buffer by calling `inputBuffer.take()`.
 - v. Check whether the retrieved robot has the value `null`.
 - A. If so, print a message that there are no more robots.
 - B. Break the loop using the `break` statement.
 - vi. Occupy the station by calling `stage.acquire(robot)`.
 - vii. Process the robot in a `try-finally` block.

- A. In the `try` block, call `stage.process(robot)`.
 - B. In the `finally` block, call `stage.release(robot)` to release the station regardless of the processing result.
- viii. Check whether `outputBuffer` is not equal to `null`.
 - A. If the output buffer exists, pass the robot further by calling `outputBuffer.put(robot)`.
 - B. If the output buffer does not exist, calculate the total production time as the difference between the current system time and `robot.getCreatedAt()`.
 - C. Print a message informing that robot production has been completed and showing the total time in milliseconds.
- ix. Handle the `InterruptedException`.
 - Print a message informing that the worker's work has been interrupted.
 - Restore the current thread interruption status by calling `Thread.currentThread().interrupt()`.
- x. After the catch block ends, print a message informing that the worker has finished.
- (g) Implement the `stopWorker()` method, whose task is to stop the worker.
 - i. Set the `running` field to `false`.
 - ii. Call the `interrupt()` method on the current thread to interrupt possible waiting.
- 6. Declare a class `AssemblyLine`, which represents a complete robot production line consisting of stages, inter-stage buffers, and workers handling the consecutive stations.
 - (a) Add private immutable fields `welding`, `electronics`, `software`, and `qualityControl` of type `Stage`, which represent the consecutive production stages.
 - (b) Add private immutable fields `inputBuffer`, `bufferWeldElectr`, `bufferElectrSoft`, and `bufferSoftQC` of type `Buffer`, which represent the input buffer and buffers between consecutive stages.
 - (c) Add a private immutable field `workers` of type `List<StageWorker>`, which stores workers handling the stations.
 - (d) Add a private field `totalRobotsCreated` of type `int`, which stores the number of robots introduced onto the line.
 - (e) Add a private immutable field `startTime` of type `long`, which stores the production line start time.
 - (f) Implement the constructor `AssemblyLine()`.
 - i. Create the stages `welding`, `electronics`, `software`, and `qualityControl`, passing their names and minimum and maximum processing times.
 - ii. Create the input buffer `inputBuffer` and the inter-stage buffers `bufferWeldElectr`, `bufferElectrSoft`, and `bufferSoftQC`, setting each capacity to 2.
 - iii. Initialize the `workers` list as a new instance of `ArrayList`.
 - iv. Store the current system time in the `startTime` field by calling `System.currentTimeMillis()`.

- v. Call the `initializeWorkers()` method to create workers for the individual stages.
- 7. Implement the private method `initializeWorkers()`, whose task is to configure the production line workers.
 - (a) Add the worker "Welder-1" handling the `welding` stage, retrieving robots from `inputBuffer` and passing them to `bufferWeldElectr`.
 - (b) Add the worker "Technician-1" handling the `electronics` stage, retrieving robots from `bufferWeldElectr` and passing them to `bufferElectrSoft`.
 - (c) Add the worker "Developer-1" handling the `software` stage, retrieving robots from `bufferElectrSoft` and passing them to `bufferSoftQC`.
 - (d) Add the worker "Inspector-1" handling the `qualityControl` stage, retrieving robots from `bufferSoftQC` and completing production without an output buffer.
- 8. Implement the `start()` method, whose task is to start the entire production line.
 - (a) Print a header in the console informing that the robot production line is starting.
 - (b) Iterate over the `workers` list and start each thread by calling `start()`.
- 9. Implement the `addRobot(Robot robot)` method, whose task is to introduce a new robot onto the production line.
 - (a) Declare that the method may throw an `InterruptedException`.
 - (b) Increase the value of the `totalRobotsCreated` field by one.
 - (c) Print a message informing that a new robot is entering the line and showing the current number of created robots.
 - (d) Add the robot to the input buffer by calling `inputBuffer.put(robot)`.
- 10. Implement the `shutdown()` method, whose task is to shut down the production line in a controlled way after all robots have been processed.
 - (a) Declare that the method may throw an `InterruptedException`.
 - (b) Print a message informing that the line shutdown procedure is starting.
 - (c) Close `inputBuffer` so that the first worker finishes work after emptying the buffer.
 - (d) Wait for the first worker to finish by calling `workers.get(0).join()`.
 - (e) Print a message informing that the `Welder` worker has finished.
 - (f) Close `bufferWeldElectr` so that the second worker finishes after processing items from the buffer.
 - (g) Wait for the second worker to finish by calling `workers.get(1).join()`.
 - (h) Print a message informing that the `Technician` worker has finished.
 - (i) Close `bufferElectrSoft` so that the third worker finishes after processing items from the buffer.

- (j) Wait for the third worker to finish by calling `workers.get(2).join()`.
 - (k) Print a message informing that the `Developer` worker has finished.
 - (l) Close `bufferSoftQC` so that the fourth worker finishes after processing items from the buffer.
 - (m) Wait for the fourth worker to finish by calling `workers.get(3).join()`.
 - (n) Print a message informing that the `Inspector` worker has finished.
 - (o) Call the `printSummary()` method to show the final production summary.
11. Implement the private method `printSummary()`, whose task is to display the final report from the production line execution.
- (a) Calculate the total working time as the difference between the current system time and the value of the `startTime` field.
 - (b) Display the production summary section header.
 - (c) Display the number of produced robots based on the `totalRobotsCreated` field.
 - (d) Display the number of robots processed in the `WELDING` stage by calling `welding.getProcessedCount()`.
 - (e) Display the number of robots processed in the `ELECTRONICS` stage by calling `electronics.getProcessedCount()`.
 - (f) Display the number of robots processed in the `SOFTWARE` stage by calling `software.getProcessedCount()`.
 - (g) Display the number of robots processed in the `QUALITY_CONTROL` stage by calling `qualityControl.getProcessedCount()`.
 - (h) Display the total working time in milliseconds.
 - (i) Display the bottom border closing the summary.
12. Declare a class `Main`, which starts the robot production line simulation.
- (a) Add a private static constant `NUMBER_OF_ROBOTS`, which specifies the number of robots created in the simulation.
 - (b) Add a private static constant `ROBOT_ENTRY_DELAY_MIN`, which specifies the minimum delay between introducing consecutive robots.
 - (c) Add a private static constant `ROBOT_ENTRY_DELAY_MAX`, which specifies the maximum delay between introducing consecutive robots.
 - (d) Implement the `main(String[] args)` method, which creates the production line, starts it, manages adding robots, and ends the program.
 - i. Create an `AssemblyLine` object and assign it to the variable `assemblyLine`.
 - ii. Start the production line by calling `assemblyLine.start()`.
 - iii. Create a `manager` thread, whose task is to create robots and pass them onto the production line.
 - A. In the `Thread` constructor, pass a lambda expression defining the manager's work logic.

- B. Name the thread `"Manager"`.
- C. Inside the lambda, place the logic in a `try` block.
- D. Start a `for` loop that runs `NUMBER_OF_ROBOTS` times.
- E. In each iteration, create a new `Robot` object.
- F. Pass the robot onto the production line by calling `assemblyLine.addRobot(robot)`.
- G. Calculate a random delay for the robot entering the line.
 - Start with the value `ROBOT_ENTRY_DELAY_MIN`.
 - Add a random value from the range from 0 to `ROBOT_ENTRY_DELAY_MAX - ROBOT_ENTRY_DELAY_MIN`.
 - Use `Math.random()` and casting to `int`.
- H. Pause the manager thread for the calculated time by calling `Thread.sleep(delay)`.
- I. After the loop finishes, print a message informing that all robots have entered the production line.
- J. Handle the `InterruptedException`.
 - Print a message informing that the manager thread has been interrupted.
 - Restore the interruption status by calling `Thread.currentThread().interrupt()`.
- iv. Start the `manager` thread by calling `manager.start()`.
- v. Place waiting for the manager to finish and shutting down the line inside a `try` block.
 - A. Wait for the `manager` thread to finish by calling `manager.join()`.
 - B. After the manager finishes, call `assemblyLine.shutdown()` to shut down the production line in a controlled way.
- vi. Handle the `InterruptedException` in the `main` method.
 - Print a message informing that the main thread has been interrupted.
 - Restore the interruption status by calling `Thread.currentThread().interrupt()`.
- vii. Print the final message informing that the program has finished.

13. Check whether the program output is:

```
=====
ROBOT ASSEMBLY LINE - START
=====
[WORKER] [Welder-1] Worker started at stage: WELDING
[BUFFER] [INPUT] Buffer empty, worker is waiting...
[WORKER] [Technician-1] Worker started at stage: ELECTRONICS
[BUFFER] [WELD->ELECTR] Buffer empty, worker is waiting...
[WORKER] [Developer-1] Worker started at stage: SOFTWARE
[BUFFER] [ELECTR->SOFT] Buffer empty, worker is waiting...
[WORKER] [Inspector-1] Worker started at stage: QUALITY_CONTROL
[BUFFER] [SOFT->QC] Buffer empty, worker is waiting...

[LINE] New robot entering the line: Robot-001 (total: 1)
[BUFFER] [INPUT] Robot-001 added to buffer. State: 1/2
[BUFFER] [INPUT] Robot-001 taken from buffer. State: 0/2
[STAGE] [WELDING] >>> Robot-001 starts processing. Station occupied.
[STAGE] [WELDING] Processing Robot-001... (time: 390ms)

[LINE] New robot entering the line: Robot-002 (total: 2)
[BUFFER] [INPUT] Robot-002 added to buffer. State: 1/2
[STAGE] [WELDING] <<< Robot-001 finished processing. Station free. Total done: 1
[BUFFER] [WELD->ELECTR] Robot-001 added to buffer. State: 1/2
[BUFFER] [INPUT] Robot-002 taken from buffer. State: 0/2
[BUFFER] [WELD->ELECTR] Robot-001 taken from buffer. State: 0/2
[STAGE] [ELECTRONICS] >>> Robot-001 starts processing. Station occupied.
[STAGE] [WELDING] >>> Robot-002 starts processing. Station occupied.
[STAGE] [WELDING] Processing Robot-002... (time: 517ms)
[STAGE] [ELECTRONICS] Processing Robot-001... (time: 570ms)

[LINE] New robot entering the line: Robot-003 (total: 3)
[BUFFER] [INPUT] Robot-003 added to buffer. State: 1/2

[LINE] New robot entering the line: Robot-004 (total: 4)
[BUFFER] [INPUT] Robot-004 added to buffer. State: 2/2
[STAGE] [WELDING] <<< Robot-002 finished processing. Station free. Total done: 2
[BUFFER] [WELD->ELECTR] Robot-002 added to buffer. State: 1/2
[BUFFER] [INPUT] Robot-003 taken from buffer. State: 1/2
[STAGE] [WELDING] >>> Robot-003 starts processing. Station occupied.
[STAGE] [WELDING] Processing Robot-003... (time: 445ms)

[LINE] New robot entering the line: Robot-005 (total: 5)
[BUFFER] [INPUT] Robot-005 added to buffer. State: 2/2
[STAGE] [ELECTRONICS] <<< Robot-001 finished processing. Station free. Total done: 1
[BUFFER] [ELECTR->SOFT] Robot-001 added to buffer. State: 1/2
[BUFFER] [WELD->ELECTR] Robot-002 taken from buffer. State: 0/2
[BUFFER] [ELECTR->SOFT] Robot-001 taken from buffer. State: 0/2
[STAGE] [SOFTWARE] >>> Robot-001 starts processing. Station occupied.
[STAGE] [SOFTWARE] Processing Robot-001... (time: 478ms)
[STAGE] [ELECTRONICS] >>> Robot-002 starts processing. Station occupied.
[STAGE] [ELECTRONICS] Processing Robot-002... (time: 677ms)
```

[LINE] New robot entering the line: Robot-006 (total: 6)
 [BUFFER][INPUT] Buffer full (2/2), Robot-006 is waiting...
 [STAGE][WELDING] <<< Robot-003 finished processing. Station free. Total done: 3
 [BUFFER][WELD->ELECTR] Robot-003 added to buffer. State: 1/2
 [BUFFER][INPUT] Robot-004 taken from buffer. State: 1/2
 [STAGE][WELDING] >>> Robot-004 starts processing. Station occupied.
 [BUFFER][INPUT] Robot-006 added to buffer. State: 2/2
 [STAGE][WELDING] Processing Robot-004... (time: 466ms)
 [STAGE][SOFTWARE] <<< Robot-001 finished processing. Station free. Total done: 1
 [BUFFER][SOFT->QC] Robot-001 added to buffer. State: 1/2
 [BUFFER][ELECTR->SOFT] Buffer empty, worker is waiting...
 [BUFFER][SOFT->QC] Robot-001 taken from buffer. State: 0/2
 [STAGE][QUALITY_CONTROL] >>> Robot-001 starts processing. Station occupied.
 [STAGE][QUALITY_CONTROL] Processing Robot-001... (time: 640ms)
 [STAGE][ELECTRONICS] <<< Robot-002 finished processing. Station free. Total done: 2
 [BUFFER][ELECTR->SOFT] Robot-002 added to buffer. State: 1/2
 [BUFFER][WELD->ELECTR] Robot-003 taken from buffer. State: 0/2
 [BUFFER][ELECTR->SOFT] Robot-002 taken from buffer. State: 0/2
 [STAGE][ELECTRONICS] >>> Robot-003 starts processing. Station occupied.
 [STAGE][ELECTRONICS] Processing Robot-003... (time: 507ms)
 [STAGE][SOFTWARE] >>> Robot-002 starts processing. Station occupied.
 [STAGE][SOFTWARE] Processing Robot-002... (time: 225ms)

[MANAGER] All robots have entered the assembly line.

[LINE] Initiating assembly line shutdown...
 [BUFFER][INPUT] Buffer closed.
 [STAGE][WELDING] <<< Robot-004 finished processing. Station free. Total done: 4
 [BUFFER][WELD->ELECTR] Robot-004 added to buffer. State: 1/2
 [BUFFER][INPUT] Robot-005 taken from buffer. State: 1/2
 [STAGE][WELDING] >>> Robot-005 starts processing. Station occupied.
 [STAGE][WELDING] Processing Robot-005... (time: 527ms)
 [STAGE][SOFTWARE] <<< Robot-002 finished processing. Station free. Total done: 2
 [BUFFER][SOFT->QC] Robot-002 added to buffer. State: 1/2
 [BUFFER][ELECTR->SOFT] Buffer empty, worker is waiting...
 [STAGE][QUALITY_CONTROL] <<< Robot-001 finished processing. Station free. Total done: 1

[COMPLETE] Robot-001 finished! Total production time: 2089ms

[BUFFER][SOFT->QC] Robot-002 taken from buffer. State: 0/2
 [STAGE][QUALITY_CONTROL] >>> Robot-002 starts processing. Station occupied.
 [STAGE][QUALITY_CONTROL] Processing Robot-002... (time: 602ms)
 [STAGE][ELECTRONICS] <<< Robot-003 finished processing. Station free. Total done: 3
 [BUFFER][ELECTR->SOFT] Robot-003 added to buffer. State: 1/2
 [BUFFER][WELD->ELECTR] Robot-004 taken from buffer. State: 0/2
 [STAGE][ELECTRONICS] >>> Robot-004 starts processing. Station occupied.
 [BUFFER][ELECTR->SOFT] Robot-003 taken from buffer. State: 0/2
 [STAGE][SOFTWARE] >>> Robot-003 starts processing. Station occupied.
 [STAGE][SOFTWARE] Processing Robot-003... (time: 200ms)
 [STAGE][ELECTRONICS] Processing Robot-004... (time: 490ms)
 [STAGE][SOFTWARE] <<< Robot-003 finished processing. Station free. Total done: 3

[BUFFER] [SOFT->QC] Robot-003 added to buffer. State: 1/2
 [BUFFER] [ELECTR->SOFT] Buffer empty, worker is waiting...
 [STAGE] [WELDING] <<< Robot-005 finished processing. Station free. Total done: 5
 [BUFFER] [WELD->ELECTR] Robot-005 added to buffer. State: 1/2
 [BUFFER] [INPUT] Robot-006 taken from buffer. State: 0/2
 [STAGE] [WELDING] >>> Robot-006 starts processing. Station occupied.
 [STAGE] [WELDING] Processing Robot-006... (time: 300ms)
 [STAGE] [ELECTRONICS] <<< Robot-004 finished processing. Station free. Total done: 4
 [BUFFER] [ELECTR->SOFT] Robot-004 added to buffer. State: 1/2
 [BUFFER] [WELD->ELECTR] Robot-005 taken from buffer. State: 0/2
 [STAGE] [ELECTRONICS] >>> Robot-005 starts processing. Station occupied.
 [STAGE] [ELECTRONICS] Processing Robot-005... (time: 475ms)
 [BUFFER] [ELECTR->SOFT] Robot-004 taken from buffer. State: 0/2
 [STAGE] [SOFTWARE] >>> Robot-004 starts processing. Station occupied.
 [STAGE] [SOFTWARE] Processing Robot-004... (time: 428ms)
 [STAGE] [WELDING] <<< Robot-006 finished processing. Station free. Total done: 6
 [BUFFER] [WELD->ELECTR] Robot-006 added to buffer. State: 1/2
 [WORKER] [Welder-1] No more robots - shutting down.
 [WORKER] [Welder-1] Worker finished.
 [LINE] Welder finished work.
 [BUFFER] [WELD->ELECTR] Buffer closed.
 [STAGE] [QUALITY_CONTROL] <<< Robot-002 finished processing. Station free. Total done: 2

[COMPLETE] Robot-002 finished! Total production time: 2551ms

[BUFFER] [SOFT->QC] Robot-003 taken from buffer. State: 0/2
 [STAGE] [QUALITY_CONTROL] >>> Robot-003 starts processing. Station occupied.
 [STAGE] [QUALITY_CONTROL] Processing Robot-003... (time: 530ms)
 [STAGE] [SOFTWARE] <<< Robot-004 finished processing. Station free. Total done: 4
 [BUFFER] [SOFT->QC] Robot-004 added to buffer. State: 1/2
 [BUFFER] [ELECTR->SOFT] Buffer empty, worker is waiting...
 [STAGE] [ELECTRONICS] <<< Robot-005 finished processing. Station free. Total done: 5
 [BUFFER] [ELECTR->SOFT] Robot-005 added to buffer. State: 1/2
 [BUFFER] [WELD->ELECTR] Robot-006 taken from buffer. State: 0/2
 [STAGE] [ELECTRONICS] >>> Robot-006 starts processing. Station occupied.
 [STAGE] [ELECTRONICS] Processing Robot-006... (time: 547ms)
 [BUFFER] [ELECTR->SOFT] Robot-005 taken from buffer. State: 0/2
 [STAGE] [SOFTWARE] >>> Robot-005 starts processing. Station occupied.
 [STAGE] [SOFTWARE] Processing Robot-005... (time: 225ms)
 [STAGE] [QUALITY_CONTROL] <<< Robot-003 finished processing. Station free. Total done: 3

[COMPLETE] Robot-003 finished! Total production time: 2757ms

[BUFFER] [SOFT->QC] Robot-004 taken from buffer. State: 0/2
 [STAGE] [QUALITY_CONTROL] >>> Robot-004 starts processing. Station occupied.
 [STAGE] [QUALITY_CONTROL] Processing Robot-004... (time: 421ms)
 [STAGE] [SOFTWARE] <<< Robot-005 finished processing. Station free. Total done: 5
 [BUFFER] [SOFT->QC] Robot-005 added to buffer. State: 1/2
 [BUFFER] [ELECTR->SOFT] Buffer empty, worker is waiting...
 [STAGE] [QUALITY_CONTROL] <<< Robot-004 finished processing. Station free. Total done: 4

[COMPLETE] Robot-004 finished! Total production time: 2954ms

```

[BUFFER][SOFT->QC] Robot-005 taken from buffer. State: 0/2
[STAGE][QUALITY_CONTROL] >>> Robot-005 starts processing. Station occupied.
[STAGE][QUALITY_CONTROL] Processing Robot-005... (time: 321ms)
[STAGE][ELECTRONICS] <<< Robot-006 finished processing. Station free. Total done: 6
[BUFFER][ELECTR->SOFT] Robot-006 added to buffer. State: 1/2
[WORKER][Technician-1] No more robots - shutting down.
[WORKER][Technician-1] Worker finished.
[BUFFER][ELECTR->SOFT] Robot-006 taken from buffer. State: 0/2
[STAGE][SOFTWARE] >>> Robot-006 starts processing. Station occupied.
[STAGE][SOFTWARE] Processing Robot-006... (time: 222ms)
[LINE] Technician finished work.
[BUFFER][ELECTR->SOFT] Buffer closed.
[STAGE][SOFTWARE] <<< Robot-006 finished processing. Station free. Total done: 6
[BUFFER][SOFT->QC] Robot-006 added to buffer. State: 1/2
[WORKER][Developer-1] No more robots - shutting down.
[WORKER][Developer-1] Worker finished.
[LINE] Developer finished work.
[BUFFER][SOFT->QC] Buffer closed.
[STAGE][QUALITY_CONTROL] <<< Robot-005 finished processing. Station free. Total done: 5

[COMPLETE] Robot-005 finished! Total production time: 3047ms

[BUFFER][SOFT->QC] Robot-006 taken from buffer. State: 0/2
[STAGE][QUALITY_CONTROL] >>> Robot-006 starts processing. Station occupied.
[STAGE][QUALITY_CONTROL] Processing Robot-006... (time: 568ms)
[STAGE][QUALITY_CONTROL] <<< Robot-006 finished processing. Station free. Total done: 6

[COMPLETE] Robot-006 finished! Total production time: 3456ms

[WORKER][Inspector-1] No more robots - shutting down.
[WORKER][Inspector-1] Worker finished.
[LINE] Inspector finished work.

```

PRODUCTION SUMMARY

```

Robots produced:      6
Stage WELDING:        6
Stage ELECTRONICS:    6
Stage SOFTWARE:       6
Stage QUALITY CONTROL:6
Total working time:   4563      ms

```

```

[MAIN] Program finished.

```

14. ★ Milestone 2: present the result to the instructor